BÀI THỰC HÀNH SỐ 02
TRUYỀN THÔNG HƯỚNG THÔNG ĐIỆP VÀ TRUYỀN THÔNG THEO NHÓM TRONG HỆ THỐNG PHÂN TÁN

I. Mục tiêu

Sau buổi thực hành, sinh viên có khả năng:

- Tự thiết kế và cài đặt một hệ thống truyền thông hướng thông điệp (message-oriented middleware) với RabbitMQ, thực thi các kịch bản kiểm thử để phân biệt truyền tin bền bỉ/nhất thời và đồng bộ/không đồng bộ đã học.
- Tự thiết kế và cài đặt một ứng dụng truyền thông theo nhóm (multicast) bằng socket UDP, quan sát cách một thông điệp được gửi một lần nhưng nhiều tiến trình cùng nhận được.

II. Chuẩn bị

Kiến thức nền

Sinh viên đã học Truyền thông hướng thông điệp và Truyền thông theo nhóm trong Chương 2. Sinh viên đã hoàn thành Bài thực hành số 01 (Gọi thủ tục từ xa và Đặt tên/DNS).

Công cụ / môi trường

- RabbitMQ (cài local hoặc chạy bằng Docker cùng thư viện client tương ứng ngôn ngữ đã chọn (pika cho Python, amqplib cho Node.js, amqp-client cho Java...).
- Ngôn ngữ lập trình hỗ trợ socket UDP (Python, Java, Node.js, C/C++...).

BÀI 1 — TRUYỀN THÔNG HƯỚNG THÔNG ĐIỆP VỚI RABBITMQ

Mục tiêu: tự thiết kế, tự cài đặt một hệ thống hàng đợi thông điệp hoàn chỉnh (Producer – Queue – Consumer), sau đó tự xây dựng và thực thi một bộ kịch bản kiểm thử để quan sát các đặc tính lý thuyết: truyền tin bền bỉ/nhất thời, truyền tin đồng bộ/không đồng bộ, và cách hệ thống phân phối tải giữa nhiều consumer.

Bước 1 — Cài đặt và khởi động RabbitMQ

Sinh viên tự cài đặt RabbitMQ (native hoặc qua Docker) và xác nhận broker hoạt động qua giao diện quản trị (http://localhost:15672).

Bước 2 — Thiết kế và cài đặt Producer ("Đặt hàng")

Sinh viên viết chương trình Producer publish thông điệp JSON lên một queue đặt tên don_hang. Nội dung thông điệp tối thiểu gồm:

Tên hàm	Tham số / Nội dung
guiDonHang	maDonHang (chuỗi), danhSachSanPham (mảng chuỗi), tongTien (số thực), thoiGianGui (timestamp)
xuLyDonHang (consumer)	Nhận thông điệp, giả lập xử lý (sleep 1–3s), gửi ack thủ công

Producer phải tự đo và in ra thời điểm gửi từng thông điệp (dùng cho các kịch bản kiểm thử bên dưới).

Bước 3 — Cài đặt Consumer ("Xử lý đơn hàng")

Consumer nhận thông điệp từ queue, giả lập thời gian xử lý (sleep ngẫu nhiên 1–3 giây), in ra nội dung đã xử lý kèm thời điểm nhận, và gửi acknowledgement thủ công (manual ack) sau khi xử lý xong.

Bước 4 — Thực hiện các kịch bản kiểm thử bắt buộc

Với mỗi kịch bản, sinh viên tự thiết kế cách chạy, ghi log kết quả và chụp lại màn hình làm bằng chứng nộp kèm báo cáo.

- KB1 — Truyền tin bền bỉ (persistent): khai báo queue với durable=true và publish thông điệp với delivery_mode=2 (persistent). Tắt toàn bộ consumer, gửi 5 thông điệp từ producer, khởi động lại RabbitMQ (hoặc restart container), sau đó bật consumer lên và quan sát consumer vẫn nhận đủ 5 thông điệp.
- KB2 — Truyền tin nhất thời (transient) đối chứng: lặp lại đúng KB1 nhưng khai báo queue với durable=false và publish không đặt persistent — quan sát và ghi lại việc thông điệp bị mất sau khi RabbitMQ restart.
- KB3 — Truyền tin không đồng bộ: producer publish liên tục 20 thông điệp không chờ phản hồi xử lý; đo và so sánh thời gian producer hoàn tất việc gửi với thời gian toàn bộ 20 thông điệp được consumer xử lý xong.
- KB4 — Cân bằng tải giữa nhiều consumer: chạy đồng thời 3 tiến trình consumer cùng lắng nghe queue don_hang, gửi 15 thông điệp từ producer, ghi lại việc RabbitMQ phân phối các thông điệp cho 3 consumer (round-robin) — mỗi consumer xử lý bao nhiêu thông điệp.

BÀI 2 — TRUYỀN THÔNG THEO NHÓM (MULTICAST)

Mục tiêu: tự cài đặt một ứng dụng gửi tin theo nhóm dùng UDP multicast, quan sát trực tiếp cách một thông điệp được gửi đi một lần duy nhất nhưng nhiều tiến trình nhận cùng nhận được nội dung đó

Bước 1 — Viết chương trình Sender

Sender mở socket UDP, gửi gói tin tới một địa chỉ multicast (ví dụ 224.1.1.1, cổng tự chọn, ví dụ 5007) mỗi 2 giây, nội dung gồm số thứ tự bản tin và timestamp gửi.

```
# Gợi ý Python – sender.py
import socket, time
MCAST_GRP = '224.1.1.1'
MCAST_PORT = 5007
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM,
                     socket.IPPROTO_UDP)
sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, 2)
seq = 0
while True:
    msg = f"seq={seq} time={time.time()}"
    sock.sendto(msg.encode(), (MCAST_GRP, MCAST_PORT))
    seq += 1
    time.sleep(2)
```

Bước 2 — Viết chương trình Receiver

Receiver join vào group multicast trên, in ra nội dung nhận được kèm thời điểm nhận thực tế (để tính độ trễ so với thời điểm gửi).

Bước 3 — Kịch bản kiểm thử

- KB1 — Nhiều tiến trình cùng nhận: chạy tối thiểu 3 receiver (trên cùng máy dùng các cổng lắng nghe khác nhau, hoặc trên 3 máy khác nhau trong cùng mạng LAN nếu có điều kiện). Xác nhận: (a) sender chỉ gửi mỗi bản tin một lần, (b) cả 3 receiver đều nhận đủ nội dung giống nhau.
- KB2 — Receiver tham gia muộn: khởi động sender trước, để chạy khoảng 20–30 giây (10–15 bản tin), sau đó mới khởi động một receiver mới. Quan sát receiver này có nhận được các bản tin đã gửi trước lúc nó tham gia hay không — đối chiếu với tính chất "nhất thời" (transient) của truyền thông theo nhóm đã học.
- KB3 (tuỳ chọn nâng cao) — Dùng Wireshark lọc theo địa chỉ multicast để quan sát trực tiếp gói tin UDP trên đường truyền, xác nhận sender chỉ tạo một luồng gói tin duy nhất dù có nhiều receiver.